Stellar Cyber 6.6.0s is now live! Discover all the updates in the release notes!

Email Action Templates

An Automated Threat Hunting (ATH) playbook monitors your environment on a schedule, runs a query against your data, and takes action when it finds results that match a condition you define. One of the actions a playbook can take is sending an email notification to alert your team that something requires attention.

This topic focuses on writing templates for the Subject and Email Body fields of an email action. An email action also includes recipients, attachments, and other configuration options, but those settings are described in the Automated Threat Hunting playbook documentation. Here you'll learn how template syntax works and how to create effective email notifications.

- Overview
- Template Syntax
- Standard Mode (Default)
- Run-for-Each Mode
- Comparison of the Two Modes
- Custom Jinja2 Filters
- Troubleshooting

Overview

When you configure an email action, you can write templates that control the content of the email—one template for the subject and another for the body. The Subject field and Email Body field can each contain plain text, template expressions, or a combination of both. If you use templates in both fields, you can use the same or different languages and Stellar Cyber will detect the language for each. The Email Body field can also be left empty if you do not want to include a message body.

Templates use placeholder variables that Stellar Cyber fills in automatically with real data from the query results at the time the playbook runs. This means you write the template once, and Stellar Cyber generates a customized email each time the playbook triggers.

When to Send an Email for Each Matching Record

Before looking at template syntax, it helps to decide how you want the email action to send notifications.

- **Send one email summarizing all matching records**. Do this when you want a single notification that provides an overview of everything the playbook found. For example: "Here are the 12 failed login attempts detected in the last hour." This is the default behavior (Standard mode; Run for each record = disabled).
- Send a separate email for each matching record. Do this when each record represents an event that should be communicated individually or sent to different recipients. For example: "Dear Joe, your account experienced a failed login from IP 10.0.0.1." This is the mode you enable when selecting **Run for each record** in the **Actions**: **Email** settings (Run-for-Each mode; Run for each record = enabled).

Each mode of behavior provides a different set of data to the template. Because of this, the template you write depends on which mode you are using. A template written for one mode doesn't work correctly in the other. This topic explains both modes, shows you the data available in each one, and provides example templates you can adapt for your own playbooks.

Template Syntax

Both modes support two template languages:

- **Mustache**: A simple, logic-less template language. Variables are enclosed in double curly braces: `{{variable_name}}`. Mustache also supports iteration (looping over a list of items) using section tags: `{{#list}} ... {{/list}}`.
- **Jinja2**: A more powerful template language that supports conditionals, loops, and filters. Variables use `{{ variable_name }}` and logic uses `{% ... %}` blocks.

You can use either language in your templates. If a template can be interpreted successfully by either language, Stellar Cyber processes it as Jinja2.

Standard Mode (Default)

Standard mode is the default behavior. It is active whenever the **Run for each record** option is not selected in the email action configuration.

How It Works

When the playbook triggers, Stellar Cyber collects all matching records from the query results and passes them to your template as a single data structure. Your template then has access to the entire set of results at once. Because all records are available together, you must write iteration logic in your template to loop through the records and display each one.

Stellar Cyber sends one email containing the rendered output of your template. All recipients receive the same email.

Data Available to Templates (Template Context)

In standard mode, the subject template and email body template has access to the full Interflow record. The following are the key parts of this record that you can reference in your templates:

Variable Path	Description
`ctx.payload.filtered`	list (array) of all matching records. Each item in this list represents one event record. This is the primary data you iterate over in your template.
`ctx.payload.filtered_total`	The total number of matching records. Useful for displaying a count in the subject line or body.
`ctx.payload.hits.total.value`	The total number of hits from the underlying query (before filtering).
`ctx.trigger.triggered_time`	The timestamp when the playbook triggered.
`ctx.trigger.scheduled_time`	The timestamp when the playbook was scheduled to run.
`ctx.payload.aggregations`	If the playbook includes calculations, this object contains the aggregation results.

Each item inside `ctx.payload.filtered` is a record object with the following structure:

- `_index`: The index where the record is stored.
- `_type`: The document type.
- `_source`: The actual event data fields (for example, `user`, `srcip`, `event_type`, `event_score`). When referencing a field inside a record, you access it as `_source.fieldname`.

Template Examples

Mustache Example

These templates produce a single email with a subject identifying the total number of detected alert events and the email body listing all detected events in bulleted format.

Subject

```
Alert: {{ctx.payload.filtered_total}} events detected
```

Email Body

```
The following {{ctx.payload.filtered_total}} events were detected at
{{ctx.trigger.triggered_time}}:

{{#ctx.payload.filtered}}
- User: {{_source.user}}, Event: {{_source.event_type}}, Source IP:
{{_source.srcip}}
{{/ctx.payload.filtered}}
```

How this works:

- `{{ctx.payload.filtered_total}}` inserts the count of matching records.
- `{{#ctx.payload.filtered}}` begins a loop over all matching records.
- Inside the loop, `{{_source.user}}`, `{{_source.event_type}}`, and `{{_source.srcip}}` pull field values from each record.
- `{{/ctx.payload.filtered}}` ends the loop.

Jinja2 Example

These templates use Jinja2 to add conditional logic—in this case, labeling events by severity based on their score—to identify the total number of alert events requiring review in the subject and present them in the email body.

Subject

```
Security Alert: {{ ctx.payload.filtered_total }} events require review
```

Email Body

```
Playbook triggered at {{ ctx.trigger.triggered_time }}.

{% for hit in ctx.payload.filtered %}
{% if hit._source.event_score >= 75 %}[CRITICAL]
{% elif hit._source.event_score >= 50 %}[MAJOR]
{% elif hit._source.event_score >= 25 %}[MINOR]
{% else %}[NOTICE]
{% endif %} User: {{ hit._source.user }}, Score: {{ hit._source.event_score }}, IP:
{{ hit._source.srcip }}
{% endfor %}
```

How this works:

- `{% for hit in ctx.payload.filtered %}` loops through each record, assigning it to the variable `hit`.
- The `{% if %}` / `{% elif %}` / `{% else %}` block checks the event score and inserts a severity label accordingly.
- Field values are accessed as `hit._source.fieldname`.

Features Available in Standard Mode

- **Attachments**: You can attach Interflow data (JSON), CSV calculation results, and JSON calculation results to the email. These options are configured in the email action settings.
- **Recipients**: You specify recipients using static email addresses or saved recipient references. Dynamic per-record recipients are not available in this mode.
- **No email throttle**: Because only one email is sent per execution, there is no per-execution email limit.

Run-for-Each Mode

Run-for-each mode is activated by selecting the **Run for each record** option in the email action configuration.



The **Run for each record** option is available only for playbooks that do not use calculations.

How It Works

Instead of passing all records to a single template rendering, Stellar Cyber processes each matching record individually. For every record that the query returns, Stellar Cyber renders the template using the data in that record and sends a separate email. This means that if the query returns 10 matching records, Stellar Cyber sends 10 emails—each one customized with the data from its corresponding record.

Because each template rendering receives only one record, you do not need to write any iteration logic. You simply reference the fields you want directly.

Data Available to the Template (Template Context)

In run-for-each mode, the template context is built from a single record. Stellar Cyber takes `_source` fields in the record and "flattens" them to the root level of the context, meaning you can access them directly by field name without any prefix. For convenience, the fields are also accessible under `_source.fieldname`.

For example, given a record like this—

```
{
  "_id": "record_123",
  "_index": "aella-ser-2025.01.01-",
  "_source": {
    "user": "john",
    "email": "john@example.com",
    "event_type": "login_failure",
    "severity": "high",
    "srcip": "10.0.0.1"
  }
}
```

—the template context becomes:

Access Pattern	Example	Result
Direct (recommended)	`{{user}}`	john
Via _source prefix	`{{_source.user}}`	john
Record metadata	`{{_id}}`	record_123

Both access patterns produce the same result. The direct pattern (without `_source.`) is recommended for readability.

Template Examples

Basic Notification

Subject

```
Security Alert for {{user}}
```

Body Email

```
Hello {{user}},

A {{severity}}-severity event was detected on your account:

  Event type: {{event_type}}
  Source IP:  {{srcip}}

Please review this activity. If you do not recognize it, contact your security team
immediately.
```

How this works:

- There is no loop. The template references fields directly because it renders against a single record.
- `{{user}}`, `{{severity}}`, `{{event_type}}`, and `{{srcip}}` are all fields from `_source` in the record, made available at the root level.

Using Jinja2 Filters

Subject

```
[{{ event_score | to_severity(uppercase=true) }}] Alert for {{ user }}
```

Body Email

```
Event detected at {{ timestamp | ts_format(hour_shift=-5, format="%Y-%m-%d
%H:%M:%S") }} (EST):

  User:     {{ user }}
  Type:     {{ event_type }}
  Severity: {{ event_score | to_severity(capitalize=true) }}
  Source:   {{ srcip }}
```

How this works:

- `| to_severity` converts a numeric event score to a human-readable severity label (see **Custom Jinja2 Filters** below).
- `| ts_format` formats a millisecond timestamp into a readable date/time string with an optional timezone offset.

Dynamic Recipients

One of the most powerful features of run-for-each mode is the ability to send each email to a different recipient based on data in the record itself. In the **Recipients** field of the email action configuration, you can use Mustache template variables that resolve per record.

For example:

- `{{_source.email}}` sends the email to the address stored in the `email` field in the the record.
- `{{user}}@yourdomain.com` constructs an email address from the `user` field.
- `admin@yourdomain.com` is a static address (always receives the email regardless of record content).

You can combine dynamic and static recipients. If a dynamic recipient fails to resolve for a particular record (for example, the `email` field is empty), Stellar Cyber skips that recipient for that record and logs the miss.

Features Available in Run-for-Each Mode

- **No iteration needed**: Each record gets its own email automatically.
- **Dynamic recipients**: Recipient addresses can be resolved from record fields using template variables.
- **Email throttle**: A maximum of 100 emails are sent per playbook execution. If the query returns more than 100 records, only the first 100 generate emails.
- **No attachments**: File attachments (interflow JSON, CSV, calculation JSON) are not supported in this mode.

Comparison of the Two Modes

The following table summarizes the key differences between standard mode and run-for-each mode to help you choose the right one for your use case:

Feature	Standard Mode	Run-for-Each Mode
Emails sent per execution	1 (summarizes all records)	1 per matching record (up to 100)
Template context	Full interflow object (`ctx.*`)	Fields in a single record (flattened to root)
How to access a field	`{{_source.user}}` inside a loop over `ctx.payload.filtered`	`{{user}}` or `{{_source.user}}` directly
Iteration required in template	Yes, you must loop through records	No, each record is handled automatically
Dynamic per-record recipients	Not supported	Supported (via template variables)
Attachments	Supported (interflow JSON, CSV, calculation JSON)	Not supported
Calculations	Compatible	Not compatible (option is hidden when calculations are present)
Email throttle	None	100 emails per execution

Custom Jinja2 Filters

The following custom Jinja2 filters are available in both standard mode and run-for-each mode. They provide convenient formatting for common data types found in security event records.

ts_format

This filter formats a millisecond-precision timestamp into a human-readable date and time string. You can optionally shift the time by a number of hours (to adjust for timezone) and specify a custom format string.

```
{{ timestamp | ts_format }}
  -> "2025-01-15 08:30:00 UTC"

{{ timestamp | ts_format(hour_shift=-5) }}
  -> "2025-01-15 03:30:00 UTC"

{{ timestamp | ts_format(hour_shift=0, format="%Y-%m-%d %H:%M:%S") }}
  -> "2025-01-15 08:30:00"
```

For a complete list of format codes, refer to strftime.org.

to_severity

This filter converts a numeric event score (0–100) into a severity label. This is useful for making email notifications immediately understandable without requiring the reader to interpret a numeric score.

Score Range	Severity Label
75–100	critical
50–74	major
25–49	minor
0–24	notice

Options:

```
{{ event_score | to_severity }}                      -> "major"
{{ event_score | to_severity(capitalize=true) }}     -> "Major"
{{ event_score | to_severity(uppercase=true) }}      -> "MAJOR"
```

Troubleshooting

- **Email not sent for a record (run-for-each mode)**: If a variable in the email body fails to resolve for a particular record (because the referenced field is missing or empty in that record), Stellar Cyber skips sending an email for that record and logs the skip under **AUTOMATION | Action History | Email Actions**.
- **Recipient resolution failures (run-for-each mode)**: If a dynamic recipient template resolves to an empty string or an invalid email format, Stellar Cyber skips that recipient and logs the reason. Check the **AUTOMATION | Action History** for details on resolution misses.
- **Attachment too large**: In standard mode, if an Interflow attachment exceeds the size limit of the email server, Stellar Cyber sends the email without the attachment and logs the following message: `Attachment too large (Email server returned 500, message: None), sent without Interflow attachment.`
- **Verifying template output**: Use the **Run Now** option on the playbook and then check **… | Last Status** to see whether the email action executed successfully and review any error details.

Common Mistakes

The most frequent source of confusion is using a template written for one mode in the other mode. Here are the symptoms and how to fix them:

Symptom	Likely Cause
Variables like `{{ctx.payload.filtered}}` or `{{ctx.payload.filtered_total}}` render as blank in the email.	You are using a standard-mode template with **Run for each record** enabled. In run-for-each mode, the `ctx` object is not available. Switch to direct field references like `{{user}}`.
Variables like `{{user}}` or `{{srcip}}` render as blank in the email.	You are using a run-for-each template in standard mode (without **Run for each record** enabled). In standard mode, fields are nested inside records in `ctx.payload.filtered`. You must iterate and use `{{_source.user}}` inside the loop.
The email body shows the iteration tags literally (for example, you see `{{#ctx.payload.filtered}}` as text in the email).	A syntax error in the Mustache template is preventing it from rendering. Check that opening and closing section tags match exactly and that variable paths are spelled correctly.

© 2026 Stellar Cyber . All rights reserved.